



Algorithms and Complexity 2026

Exercise Session 7

This is a collection of problems that hopefully improve your understanding of the material discussed in the lectures. During the exercise classes you have time to work on the first 3-5 problems and will have the opportunity to obtain feedback and/or help from one of the teaching assistants. Towards the end of each session the solutions of these problems will be covered.

To optimally prepare for the mastery test, you are encouraged to solve some of the remaining problems on your own. If you are stuck, require assistance or someone really messed up to write an understandable solution, feel free to send an email to algokomp-help@kth.se or visit an office hour.

Some problems are marked with superscripts: $1^{\mathbf{M}}$, $2^{\star}$ or even $3^{\star\star}$. Problems marked with an $\mathbf{M}$ superscript are in the spirit of mastery test problems. Such problems are usually not stated as plain math problems but rather consist of a story from which you need to extract the mathematical structure yourself. Problems marked with a $\star$ are challenging problems. The more stars the harder – do not get discouraged if you cannot solve such a problem.

1 Knapsack: decision, optimization and search

We are given objects $1, 2, \dots, n$, where object i is of integer weight w_i , and integer value v_i . Further we are given a bag that can carry any set of objects of weight up to C . We want to fill our bag with objects of maximum value.

Let $\text{KNAPSACK}(w_1, \dots, w_n, v_1, \dots, v_n, C, K)$ return 1 iff there is a set of objects $S \subseteq \{1, \dots, n\}$ such that the total weight of S does not exceed the capacity of the bag $\sum_{i \in S} w_i \leq C$, while the total value of the objects in S is at least K . That is, $\sum_{i \in S} v_i \geq K$. Otherwise KNAPSACK returns 0.

Suppose we are given an algorithm solving above problem in time $T(n)$. For the following tasks let us fix $U = \max\{\sum_{i=1}^n w_i, \sum_{i=1}^n v_i\}$.

- For a given set of objects and a bag, come up with an algorithm to determine the maximum value that the bag can carry. The time complexity of your algorithm should be $O(\log(U) \cdot T(n))$.
- Given a set of objects and a bag, come up with an algorithm to determine the objects that maximize the value that can be carried in the bag. Your algorithm should run in time $O(P(\log(U), n) \cdot T(n))$, where $P(\log(U), n)$ is a polynomial.

Solution:

- For ease of notation let us also denote by $W = \{w_1, \dots, w_n\}$ the weight vector of length n , and similarly by V the value vector.

Notice that $0 \leq K \leq \sum_i v_i$, and that if, for some K_0 , we have $\text{KNAPSACK}(W, V, C, K_0) = 1$, then for any $K \geq K_0$, $\text{KNAPSACK}(W, V, C, K) = 1$. Hence, we can perform a binary search in $\{0, \dots, \sum_i v_i\}$ to find the largest value of K such that the decision algorithm still outputs 1. This runs in $O(\sum_i v_i \cdot T(n))$.

- b) The main idea is to use the decision algorithm to determine if each of the item is necessary to reach an optimal solution:

Algorithm 1

```

1: function CONSTRUCTKAPSACK( $W, V, C$ )
2:    $K \leftarrow$  max integer such that  $\text{KNAPSACK}(W, V, C, K) = 1$  ▷ use binary search
3:    $S \leftarrow \{1, 2, \dots, n\}$ 
4:   for  $i \in \{1, 2, \dots, n\}$  do ▷ Try to remove element  $i$ 
5:     if  $\text{KNAPSACK}(W \setminus \{w_i\}, V \setminus \{v_i\}, C, K)$  then ▷ Can we still reach value  $K$  without  $i$ ?
6:        $S \leftarrow S \setminus \{i\}$ 
7:        $W \leftarrow W \setminus \{w_i\}$ 
8:        $V \leftarrow V \setminus \{v_i\}$ 
9:   return  $S$ 

```

The algorithm calls KNAPSACK at most $n + \log(\sum_i v_i)$ times, which yields a runtime of $O((n + \log(U)) \cdot T(n))$ as required. Now, we need to prove that the returned set S is an optimal solution.

We first show that S contains an optimal solution. Let $\mathcal{S}^*$ be the set of all optimal solutions to the instance (W, V, C) . For a set S , let $W_S = \{w_i, i \in S\}$ denote the set of weights of items in S , and similarly, $V_S = \{v_i, i \in S\}$. For clarity, we write S_i for the state of S before loop i .

We will prove our claim using the following loop invariant: “For all $i \in 1, \dots, n + 1$, there exists $S^* \in \mathcal{S}^*$ such that $S^* \subseteq S_i$ ”.

Initialization: Before the loop, every item is in S_1 , hence, for any $S^* \in \mathcal{S}^*$, we have $S^* \subseteq S_1$.

Maintenance: Suppose that the loop invariant holds before loop i . We will show that it holds after loop i . By loop invariant, we know that there exists $S^* \in \mathcal{S}^*$ such that $S^* \subseteq S_{i-1}$. Now, either $\text{KNAPSACK}(W_S \setminus \{w_i\}, V_S \setminus \{v_i\}, C, K) = 0$. In that case, $S_i = S_{i-1}$ and thus $S^* \subseteq S_i$. Otherwise, $\text{KNAPSACK}(W_S \setminus \{w_i\}, V_S \setminus \{v_i\}, C, K) = 1$. In that case, we remove item i and we set $S_i = S_{i-1} \setminus \{i\}$. But then, since $\text{KNAPSACK}(W_S \setminus \{w_i\}, V_S \setminus \{v_i\}, C, K) = 1$, we know that the instance is satisfiable, and thus that there exists $S'^* \in \mathcal{S}^*$ (that could be different from S^*) such that $S'^* \subseteq S_i$.

Termination: when the loop is terminated, the loop invariant shows that there exists $S^* \in \mathcal{S}^*$ optimal solution such that $S^* \subseteq S$.

Finally notice that S contains no unnecessary elements, as if we remove any item from S , we cannot reach optimal value K anymore.

Hence, at the end of the algorithm, S is an optimal solution to the input instance.

2 Positive reduction

A useful way to solve problems is by finding a reduction to a problem which you already know how to solve. You should use this method to solve the following problem.

INPUT: A connected undirected graph $G = (V, E)$ and a positive integer K between 1 and $|V|$.

PROBLEM: Is it possible to remove K edges from graph G to make it disconnected (ie. divided into connected components)?

Solution:

Suppose X is the minimal number of edges whose removal makes G disconnected. (This means that no strict subset of X makes G disconnected.) Then G consists of two components, and all edges of X go between these two components. (Otherwise, X can not be minimal.) Thus, X corresponds to a cut in the graph (a division of the vertices in two parts). The number of edges in X is the cut size. This means that the minimum number of edges that we must remove so that G becomes disconnected — call this $\lambda(G)$ — is equal to the size of a minimum cut (V_1, V_2) in G .

Minimum cut is the same as the maximum flow. We shall therefore try to reduce our problem to a maximum flow between two vertices s and t in a graph. We extend G to a flow graph G' , by giving each bidirectional edge capacity 1 and find $\lambda(G)$ by calculating the maximum flow from s to t for different s and t . However, we don't need to vary both. If we choose an arbitrary s , it must belong to one of the sets V_1 and V_2 above. Varying t over all vertices in addition to s , we are guaranteed to reach a vertex in the second set. Therefore, the answer is an arbitrary vertex s .

$$\lambda(G) = \min_{t \in V - \{s\}} \{\text{MaxFlow}(G', s, t)\}.$$

Now it remains to determine if $K \geq \lambda(G)$. We answer NO if $K < \text{MaxFlow}(G', s, t)$ for all $t \in V - \{s\}$ and YES otherwise.

The flow algorithm is called $|V| - 1$ times. Each run takes time $O(|V|^3)$. So then, the complexity of our algorithm is $O(|V|^4)$.

3 Scheduling: decision, optimization and search

We are given jobs $1, \dots, n$ and machines $1, \dots, m$, where job i takes integer time t_i to complete when run on a machine. We want to schedule the jobs to the machines such that the time until all jobs are completed is minimized.

Let $\text{SCHEDULE}(t_1, \dots, t_n, m, C)$ return 1 iff the jobs can be schedule onto the m machines such that all jobs are completed in time C .

Suppose we are given an algorithm solving above problem in time $T(n)$. For the following tasks let us fix $U = \sum_{i=1}^n t_i$.

- a) For a given set of jobs and machines, come up with an algorithm to determine the minimum time required to finish all jobs. The time complexity of your algorithm should be $O(\log(U) \cdot T(n))$.
- b) Given a set of jobs and machines, come up with an algorithm to determine how to schedule the jobs onto the machines such that the time required to finish all jobs is minimized. Your algorithm should run in time $O(P(\log U, n) \cdot T(n))$, where $P(\log U, n)$ is a polynomial.

Solution:

- a) Binary search over the parameter C to find the smallest value such that the decision algorithm still outputs 1. Note that C is at most $\sum_i t_i$ and hence the algorithm runs in the appropriate time.

- b) First run the algorithm from the previous task to obtain an integer C which is the smallest number such that all jobs can be scheduled in that time.

We then recursively try to merge the first job with some other job, that is, for each $i \in \{2, \dots, n\}$ we create a merged job $1i$, running in time $t_{1i} = t_1 + t_i$, and check whether

$$\text{SCHEDULE}(t_{1i}, t_2, \dots, t_{i-1}, t_{i+1}, \dots, t_n, m, C) = 1.$$

If this holds for some i , then we know that job 1 and i can be scheduled on the same machine and we recursively resolve the smaller problem with the $n - 1$ remaining jobs

$$t_{1i}, t_2, \dots, t_{i-1}, t_{i+1}, \dots, t_n.$$

If there is no other job to merge job 1 with, then we know that it has to run on its own machine. Hence we can schedule job 1 on machine 1 and recursively schedule the remaining $n - 1$ jobs on the other $m - 1$ machines.

Note that when running the algorithm we need to keep track of which jobs have been merged so far to extract a schedule in the end. This can be done by maintaining a list for each job that contains all original jobs which have been merged into the current job. With these lists one can then recover a schedule.

Regarding the running time, we claim that at most n^2 many merges are performed. Consider the parameter n . Note that in each recursive call it is reduced by 1 and in each recursion we perform at most n merges. Each such merge results in a call to the SCHEDULE algorithm. Hence this part of the algorithm runs in time $O(n^2 \cdot T(n))$.

The overall running time with the optimization for C is $O((n^2 + \log U) \cdot T(n))$, as required.

4 Negative reduction

Bin packing is the following problem. You are given n objects, each weighing between 0 and 1 kg, and some boxes to place the objects in. The goal is to find the minimum number of boxes needed to store the n objects such that no box contains more than 1 kg.

This is a well-known problem which is difficult to solve exactly (it's a so called *NP-complete problem*). As we do not expect to solve this problem optimally in polynomial time, your friend devised the following FIRSTFIT heuristic:

Suppose the objects as well as the boxes are indexed by an integer. Take the object with the least index not yet stored and place it into the box with the smallest index that has capacity for this object (i.e., if the object weighs w kg, then store it in the first box that contains at most $1 - w$ kg).

Your friend claims that they managed to implement the FIRSTFIT heuristic in $O(n)$ time complexity, only using comparisons. You doubt their claim and want to prove that this cannot be.

Show that $\Omega(n \log n)$ is a time complexity lower bound for the FIRSTFIT heuristic if only comparisons are used.

Solution:

As usual for lower bounds in $\Omega(n \log n)$, we construct a reduction of the problem of sorting n numbers to our problem. We know it is impossible to sort n numbers by means of comparisons faster than $\Omega(n \log n)$. This applies even if the n numbers are permuted integers from 1 to n , and even if we allow to make an initial linear re-scaling of the numbers. Let us show how we can use FIRSTFIT to sort these numbers.

The idea is to first ensure that the i th box, for $1 \leq i \leq n$, has capacity $\frac{1}{n-i+1}$. Then, if we are given a permutation of the fractions $1, 1/2, \dots, 1/n$ the FIRSTFIT heuristic will place the fraction $\frac{1}{i}$ into box $n - i$ and hence sort these in increasing order.

Algorithm 2 Given a permutation of $1, 1/2, 1/3, \dots, 1/n$ this algorithm returns them in increasing order.

```

1: function SORT( $v_1, \dots, v_n$ )
2:   for  $i \leftarrow 1, 2, \dots, n$  do
3:      $x_i \leftarrow 1 - \frac{1}{n-i+1}$  ▷ FIRSTFIT will put  $1 - \frac{1}{n-i+1}$  kg into the  $i$ th box
4:   for  $i \leftarrow 1, 2, \dots, n$  do
5:      $x_{n+i} \leftarrow v_i$ 
6:    $L \leftarrow \text{FIRSTFIT}(x_1, \dots, x_{2n})$ 
7:   for  $i \leftarrow 1, 2, \dots, n$  do
8:      $r_i \leftarrow L_i[2]$  ▷ Take the second object out of each box
9:   return  $r_1, \dots, r_n$ 

```

Function Sort reduces the sorting problem to FirstFit. Reduction (not counting the call to FirstFit) takes time $O(n)$, so if you could implement FirstFit in time less than $\Omega(n \log n)$, it would be possible to sort n numbers faster than $\Omega(n \log n)$, which is impossible.

5 Undecidability 1

Is there an explicit program P so that for a given y it is decidable whether P terminates on input y ?

Solution:

Yes, there are plenty of such programs. Take for example the simple program $P(y) = \text{return}$. It is easy to see that P terminates on all input.

6 Undecidability 2

Is there an explicit program P so that for a given y it is undecidable whether P terminates on input y ?

Solution:

Yes. Let P be an interpreter, and let the input $y = xy'$, where x is a program and y' is an input to that program. This means that $P(y)$ behaves in the same way as the program x on input y' , and it is undecidable whether a program x terminates on a specific input y .

7 Undecidability 3

If the program x terminates on empty input, we denote by $f(x)$ the number of steps before terminating. Otherwise, we set $f(x) = 0$. Define now $MR(y)$, the maximum running time of all programs whose binary encoding is less than y , i.e.,

$$MR(y) = \max_{x \leq y} f(x).$$

Is MR computable?

Solution:

No, MR is not computable. We know it's undecidable whether a program terminates on the empty input (empty string). Therefore reduce this problem to MR . Note that $MR(y)$ is the maximum number of steps that each program of "size" at most y need before terminating, if it starts with an empty input.

Algorithm 3 With the help of MR , this algorithm decides whether the machine y terminates given an empty input.

```

1: function STOPBLANK( $y$ )
2:    $s \leftarrow MR(y)$ 
3:   simulate  $y$  on the empty input for  $s$  steps (or until  $y$  terminates)
4:   if  $y$  terminated then
5:     return True
6:   else
7:     return False

```

If y doesn't terminate within s steps, we know that it will never terminate. Since StoppBlank is not computable, MR can not be computable either.

8 Undecidability 4

Show that the function MR in the previous exercise grows faster than any computable function. In particular, show that for every computable function g there is a y such that $MR(y) > g(y)$.

Solution:

Suppose the opposite, namely that there is a computable function g such that $g(y) \geq MR(y)$ for all y . Then we could replace the first instruction $s \leftarrow MR(y)$ in the solution to the previous exercise with the instruction $s \leftarrow g(y)$ and the program will still work. In addition, the program will be computable, but we know it can not be so. Therefore, we have a contradiction and our assumption is false.

9 Undecidability 5

Suppose you are given a Turing machine M , an input X (which is on the tape from the beginning) and an integer constant K . Is the following problem decidable or not?

Does M terminate on input X using at most K cells of the tape (a used cell may be written and read several times)?

Solution:

The problem is decidable. Since the Turing machine must remain within the K cells on the tape, there are only a finite number of configurations of the Turing machine can be in. If the machine has m states, the number of configurations is $m \cdot K \cdot 3^K$ (the number of states times the number of possible positions of the head for reading and writing times the number of possible tape configurations). Simulate a Turing machine in $m \cdot K \cdot 3^K + 1$ steps and check all the time that it does not move beyond the K cells on the tape. If the Turing machine stops within this time, we answer *yes*. If the Turing machine does not stop after this time, it must return to a configuration it has previously been in, therefore it is inside an infinite loop and we can safely say *no*.

10 Coloring: decision, optimization and search

Let G be a graph on n vertices and suppose that $\text{GRAPHCOLORING}(G, k)$ returns 1 iff the vertices of G can be properly k -colored. In other words, the algorithm returns 1 if and only if there is a coloring of the vertices of G with k colors such that the endpoints of each edge have distinct colors. Let $T(n)$ be an upper bound on the time complexity of $\text{GRAPHCOLORING}(G, k)$, for any k .

- a) Construct an algorithm that given a graph G with n vertices determines the minimum number of colors needed to color G . The time complexity should be $O(n^{1/2} \cdot T(n))$.
- b) Construct an algorithm that given a graph G with n vertices colors each vertex with the minimum number of colors in time $O(P(n) \cdot T(n))$, where $P(n)$ is a polynomial.

Solution:

- a) We know that the colors are between 1 and n . Do binary search in this interval by using the algorithm GRAPHCOLORING to find a k so that $\text{GRAPHCOLORING}(G, k) = 1$ and $\text{GRAPHCOLORING}(G, k - 1) = 0$. This procedure requires minimal coloring to have k colors. We need at most $\log n$ iterations to get down to 1. The time complexity is therefore $O(\log n \cdot T(n))$.
- b) Find the minimum number of colors k with the method from the previous assignment. Color the vertices of G with colors from 1 to k using the following algorithm:

Algorithm 4 Given a k -colorable graph and a decision algorithm GRAPHCOLORING , this algorithm constructs a proper k -coloring.

```

1: function CREATECOLORING( $G, k$ )
2:    $(V, E) \leftarrow G$ 
3:    $c : V \rightarrow \{1, 2, \dots, k\}$  ▷ Will eventually be a proper  $k$ -coloring
4:    $u \leftarrow_{\text{any}} V$ 
5:    $c(u) \leftarrow k$ 
6:    $C_k \leftarrow \{u\}$  ▷ Contains all vertices colored  $k$ 
7:   for  $v \in V \setminus \{u\}$  do ▷ Add  $(u, v)$  to the graph: still  $k$ -colorable?
8:     if  $\text{GRAPHCOLORING}((V, E \cup \{(u, v)\}), k)$  then
9:        $E \leftarrow E \cup \{(u, v)\}$ 
10:    else
11:       $c(v) \leftarrow k$ 
12:       $C_k \leftarrow C_k \cup \{v\}$ 
13:  if  $k \geq 1$  then
14:     $c \leftarrow c \cup \text{CREATECOLORING}((V \setminus C_k, E), k - 1)$ 
15:  return  $c$ 

```

By induction one can show that after the for loop, the graph (V, E) is k -colorable, and no vertex in C_k is adjacent to a vertex in C_k . Furthermore, before the recursive call, every vertex in $V \setminus C_k$ is adjacent to u . Hence all vertices in $V \setminus C_k$ must be colored by a color distinct from $c(u)$. As the graph (V, E) is k -colorable, we see that the graph $(V \setminus C_k, E)$ is $k - 1$ colorable. This shows that if G is initially a k -colorable graph then this procedure in fact returns a proper k -coloring of G .

GRAPHCOLORING is called at most once for each pair of vertices in the graph. Thus the time complexity of the algorithm is $O(\log n \cdot T(n) + n^2 \cdot T(n)) = O(n^2 \cdot T(n))$.